

Sprint report

Sprint	Sprint 1
Period	1st week, from Wednesday 26 th August
Contributor	Oussama Aaraba
Repository / tag	GitHub: https://github.com/OAoussama/blade-inspection.git sprint - 1

1. Summary

Sprint objective: build the complete application chain, from image upload to the display of detected damage.

The chain works end to end: a user uploads photos, the API stores and records them, an asynchronous job produces detections, and the interface displays them as rectangles drawn on the images.

The detection model is not trained yet. The inference service returns dummy detections in the final format. This is a deliberate choice: since the output contract is fixed, integrating YOLOv8 will only change a single function.

2. Backend

What works: a FastAPI API structured into routers, services and a storage layer. PostgreSQL database with 6 tables (`turbines`, `users`, `inspections`, `images`, `detections`, `reports`), managed by Alembic (`acc0d0368ffc2`, `cb689e7e085b`). Endpoints visible in `/docs` : `POST /inspections` (202), `GET /inspections/{id}`, `GET /images/{id}/file`, `GET /health`.

Technical decisions:

- Asynchronous inference with a 202 response and polling, since an inspection covering several images exceeds any reasonable HTTP timeout.
- Bounding box coordinates normalised to 0-1, matching YOLO native output, so no conversion has to be written and the display stays correct at any size.
- Images stored outside the database, referenced by a relative key, behind an abstract interface, so moving to MinIO will not affect any service.
- Severity computed by rules (`class x area x confidence`), explainable and adjustable.
- Configuration through environment variables, no credentials in the repository.

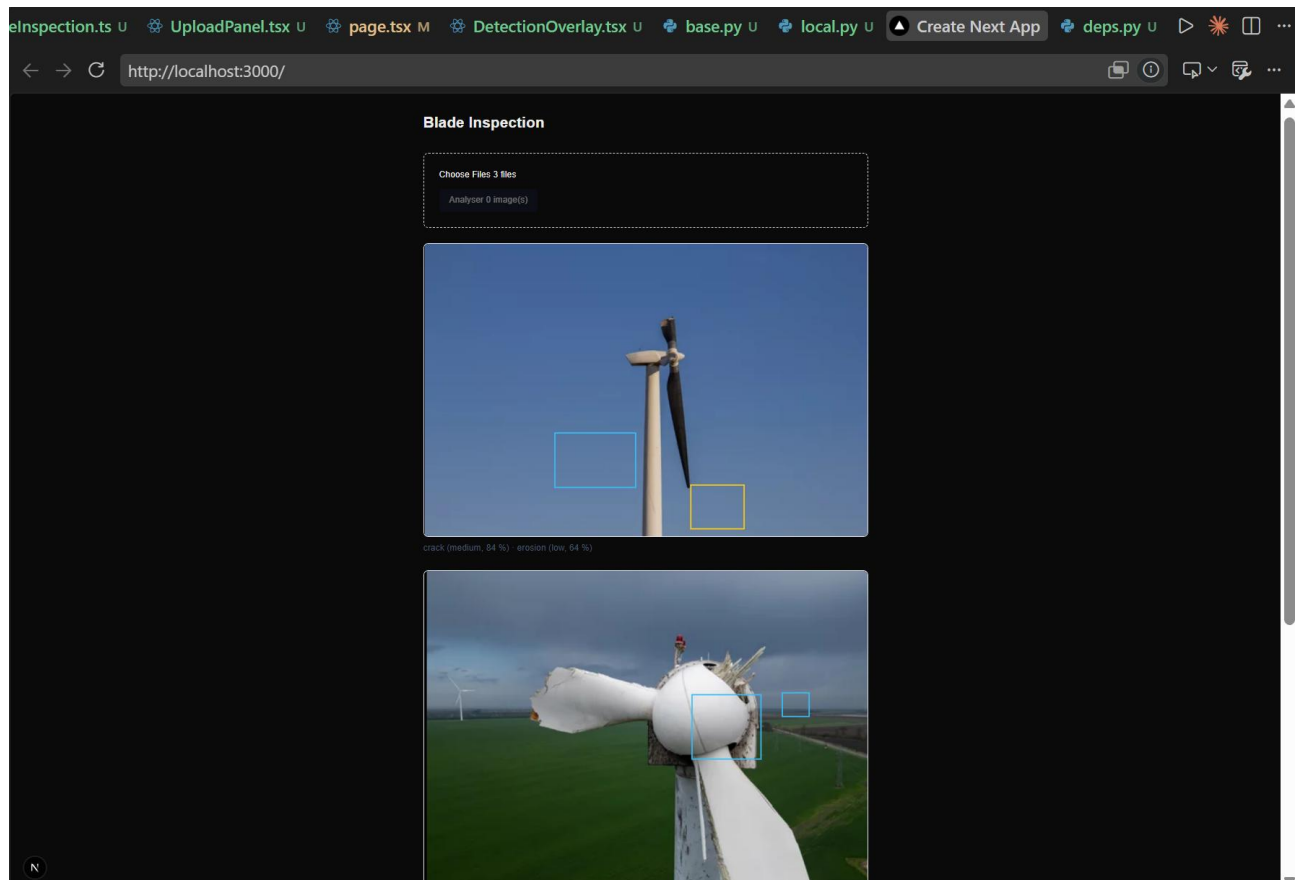
Not covered: authentication, deletion of orphaned files, PDF report generation.

3. Frontend

Next.js application with a centralised HTTP client (timeout, cancellation, typed errors). The user can select several images, launch the analysis, follow its progress, and see the damage outlined on each photo with its class and severity.

Decisions: TypeScript types aligned with the Pydantic schemas, so any contract break is caught at compile time. Polling with a recursive `setTimeout` rather than `setInterval`, to avoid requests stacking up.

Not covered: turbine selection (identifier hard-coded), per-turbine report view.



- The screenshot shows the application running locally, illustrating the response coming from the backend and displayed on the frontend with dummy detections.

4. Model

Work not started. The inference service (`stub-0.1`) returns dummy detections matching the target format: class, confidence, normalised bbox, severity.

The sequencing is intentional: building the application first fixes the format the model will have to produce. The reverse order would have forced an adaptation layer to be written afterwards.

Dataset search in progress; this is the limiting factor identified for what comes next.

5. Next sprint

- Build the annotated dataset (cracks, erosion) and check the licences
- Fine-tune YOLOv8 and replace the implementation of `predict()`
- Add `GET /turbines` and the selection on the interface side
- First version of the per-turbine report view

Open questions:

- Do you have annotated blade images, or should I work from public datasets?
- Which damage classes should be covered as a priority?
- Which shared repository should be used for sprint deliveries?